MindMate — Complete Production Deployment Guide

This guide explains how to deploy **MindMate** live on the web so you can share a live working link on your resume and portfolio.

---

1. Quick Overview of Environment Variables

You do not need to be confused by multiple `.env` files. Here is the master breakdown:

ENVIRONMENT VARIABLE	WHERE TO PUT IT	DESCRIPTION	EXAMPLE / DEFAULT
`PORT`	Backend (`server/.env`)	Port on which the API gateway runs	`5000` (or assigned by host)
`NODE_ENV`	Backend (`server/.env`)	Runtime mode	`production`
`MONGO_URI`	Backend (`server/.env`)	MongoDB Atlas connection string	`mongodb+srv://<user>:<pwd>@cluster0.abcde.mongodb.net/mindmate`
`JWT_SECRET`	Backend (`server/.env`)	Random string for signing user session tokens	`your_secret_key_minimum_32_characters`
`JWT_EXPIRES_IN`	Backend (`server/.env`)	Token expiration	`7d`
`GEMINI_API_KEY`	Backend (`server/.env` or `ai-service/.env`)	Google Gemini API Key for CalmFlow Chatbot	Get free from Google AI Studio
`FASTAPI_SERVICE_URL`	Backend (`server/.env`)	Python ML service URL (Optional)	`http://localhost:8000` or leave empty
`SMTP_HOST` , `SMTP_USER` , `SMTP_PASS`	Backend (`server/.env`)	Optional Gmail SMTP for SOS alert emails	`smtp.gmail.com`
`SERVE_STATIC`	Backend (`server/.env`)	Set to `true` if Node serves the React build directly	`true` (Unified deployment)

ENVIRONMENT VARIABLE	WHERE TO PUT IT	DESCRIPTION	EXAMPLE / DEFAULT
`VITE_API_BASE_URL`	Frontend (`client/.env`)	Base URL for API requests	`https://your-backend.onrender.com/api` (or `/api` if unified)

2. Choosing Your Deployment Architecture

Architecture Option A: Unified Full-Stack (Recommended for Simple Free Hosting)

- **Host**: **Render** (Web Service) or **Railway**
- **How it works**:
  1. Build the React frontend (`npm run build`).
  2. The Express server serves the built React app AND the API endpoints from the exact same URL.
  3. Uses **MongoDB Atlas** (Free Tier) for the database.
  4. Uses **Google Gemini API** (`GEMINI_API_KEY`) directly from the backend for the AI Companion.
- **Benefits**:
  - Only **1 service to deploy and maintain**.
  - No CORS issues because the frontend and backend share the same domain.
  - Zero cost on Render / Railway free tier.

Architecture Option B: Decoupled Microservices

- **Frontend**: **Vercel** or **Netlify** (Vite React app)
- **Backend API Gateway**: **Render** or **Railway** (Node.js Express)

- **AI Microservice**: **Render** or **Railway** (Python FastAPI)
- **Database**: **MongoDB Atlas** (Free M0 cluster)
- **Benefits**:
  - Independent scaling and automatic CDN edge deployment for the frontend.

---

3. Step-by-Step Deployment Instructions

Step 3.1: Create a Free MongoDB Database

1. Go to MongoDB Atlas and register a free account.
2. Click **Create Deployment** → Select **M0 Free Cluster**.
3. Under **Security Quickstart**:
   - Create a database user (e.g. `mindmate_admin` and a strong password).
   - Under **Network Access**, add IP `0.0.0.0/0` (Allow access from anywhere).
4. Click **Connect** → **Drivers** → Copy the connection string:

   ```
   mongodb+srv://<username>:<password>@cluster0.abcde.mongodb.net/mindmate?retryWrites=true&w=majority
   ```

   Save this as your `MONGO_URI`.

Step 3.2: Get a Free Google Gemini API Key

1. Go to Google AI Studio.
2. Sign in with your Google account.
3. Click **Get API key** → **Create API key in new project**.
4. Copy your API key (starts with `AIzaSy...`).
5. Save this as `GEMINI_API_KEY`.

Step 3.3: Deploying on Render (Option A — Unified)

1. **Push your code to GitHub**:

```
git add .
git commit -m "MindMate production ready"
git push origin main
```

2. **Create New Web Service on Render**:

   - Go to render.com → Dashboard → **New** → **Web Service**.
   - Connect your GitHub repository.
   - Configure the service settings:
     - **Name**: `mindmate-wellness`
     - **Language / Runtime**: `Node`
     - **Branch**: `main`
     - **Build Command**:

```
npm --prefix client install && npm --prefix client run build &&
        npm --prefix server install
```

     - **Start Command**:

```
cd server && node server.js
```

3. **Set Environment Variables on Render**: In the Render dashboard, navigate to **Environment** and add:

   - `NODE_ENV` : `production`
   - `PORT` : `10000`
   - `MONGO_URI` : `mongodb+srv://...` (your Atlas URI from Step 3.1)
   - `JWT_SECRET` : `your_random_jwt_secret_key_minimum_32_characters`
   - `GEMINI_API_KEY` : `AIzaSy...` (your Gemini key from Step 3.2)
   - `SERVE_STATIC` : `true`

4. Click **Deploy Web Service**! Render will build the React frontend, start Express, and give you a live URL like `https://mindmate-wellness.onrender.com`.

Step 3.4: Deploying on Vercel (Frontend) + Render (Backend) (Option B)

If you prefer hosting the React client separately on Vercel:

1. **Deploy Backend on Render**:
   - Root directory: `server`
   - Build command: `npm install`
   - Start command: `node server.js`
   - Environment variables: `MONGO_URI`, `JWT_SECRET`, `GEMINI_API_KEY`, `NODE_ENV=production`.
   - Your API will be live at: `https://your-mindmate-api.onrender.com`.
2. **Deploy Frontend on Vercel**:
   - Import your GitHub repo on vercel.com.
   - Root directory: `client`
   - Framework preset: `Vite`
   - Environment Variable:
     - `VITE_API_BASE_URL`: `https://your-mindmate-api.onrender.com/api`
   - Click **Deploy**!

---

4. Verifying Your Live Deployment

Once deployed, verify these key workflows:

1. **User Sign Up & Login**: Create an account and check that your name is saved cleanly.
2. **Profile Photo**: Upload a picture or edit details to confirm persistence.
3. **Daily Check-In**: Adjust the sliders (enforced 24-hour limit) and click **Analyze My Wellness**.
4. **CalmFlow AI Chatbot**:

- Open **AI Companion**.
- Send: *"I'm feeling anxious about finals"*.
- CalmFlow will reply empathetically taking into account your stress level and score.

5. **Multi-Layered Breathing Exercise**:
   - Click **Breathe** under recommended exercises to see the soothing concentric purple circle animations.
6. **Navigation**:
   - Use the `<` and `>` header buttons or swipe gestures to move back and forward through visited tabs.